

Program Sandbox Reference

This document describes the environment your forged programs run in, exactly as the runner executes them. Treat every statement here as ground truth about the sandbox. When a task needs something this environment does not provide, say so and ask the user for it (see "When the task needs more", below) instead of retrying variations.

What a program is

One self-contained source file in **bash**, **python**, or **node**, executed in a fresh, throwaway container on the user's VPS. The container is created for the run and destroyed afterward. Nothing carries over between runs except what the program prints — unless the program was approved with **persistence**, which grants it a durable private `/state` directory (see "Persistent state" below).

Interpreter	Version	Invoked as
bash	5.3 (Alpine)	<code>bash /work/prog</code>
python	3.12	<code>python3 /work/prog</code>
node	20	<code>node /work/prog</code>

The base system is Alpine Linux with exactly these three interpreters. There is no curl, no git, no compiler toolchain.

Resource limits (per run)

As configured on this runner (2026-08-25):

- **Memory:** 1 GB, no swap (a manifest may request up to 2 GB). **CPU:** 1 core. **Processes/threads:** 256 (requestable up to 1024).
- **Wall clock (interactive):** the manifest's `timeout_ms`, clamped to 1–60 seconds (default 15s). The container is killed shortly after the limit. Longer wall-clock exists only on a **user-set schedule** — see "Long unattended runs" below; a live run can never exceed the sync cap.
- **Open files:** 1024.
- **Output:** stdout and stderr are each captured up to **512 KB**, then truncated with `[...truncated]`. Results larger than that must be summarized or chunked across runs by design.

Input and output

- **Input** arrives as a JSON object in the environment variable `PROGRAM_ARGS`. Read it defensively — it may be absent or `{}`:

```
`python import os, json args = json.loads(os.environ.get("PROGRAM_ARGS") or "{}")`
```

- **Output** is whatever the program prints to stdout (stderr for diagnostics). The exit code matters: `0` is success. There is no other channel — no files survive the run (except `/state`, for persist programs).

Filesystem

- The filesystem is **read-only**, including the working directory `/work` (the program itself is at `/work/prog`, read-only).

- **/tmp is writable scratch**: 64 MB, wiped when the run ends, and **no-execute** — files written there cannot be executed. `/dev/shm` offers 16 MB more. Use scratch for intermediate data only; results must still be printed.
- **/state is durable storage for persist programs only** (see the next section). For everything else it is ephemeral or absent.
- Consequence: downloading or generating a binary/script and executing it is not possible. All logic lives in the one source file.

Persistent state (`/state`)

A program forged with `persist: true` in its manifest keeps a private, durable `/state` directory between runs. Facts that shape how you write one:

- `/state` belongs to the program's **lineage** (all versions of one name share the identity), but each **newly approved version starts with a fresh, empty `/state`** unless the user explicitly chooses to carry the old state forward on the approval card. Never assume prior state exists: treat a missing file as first-run and initialize it.
- The **verification smoke test always sees an empty, throwaway `/state`** — the first-run path is the verified path.
- Soft quota ~256 MB per program; store compact working state (counters, cursors, caches, small databases), not bulk archives. Results must still be **printed** — `/state` is memory, not an output channel.
- Anything written to `/state` is readable by later runs of the same lineage and by the VPS operator. Don't store secret values there.
- Declare `persist: true` only when the job genuinely needs memory between runs; stateless stays the default and the safer choice.

Libraries

- **Python stdlib + a baked-in third-party set**, Node 20 built-ins, and POSIX/BusyBox tools. The Python packages available for `import` are: `requests`, `beautifulsoup4` (`bs4`), `lxml`, `html5lib`, `aiohttp`, `yaml` (PyYAML), `dateutil`, `numpy`, and `pandas` — plus everything in the standard library. Node has a global `fetch()` for HTTP.
- **No runtime installation.** `pip/npm/apk` cannot install at run time (read-only filesystem, no-execute scratch, offline by default). **Never write a program that installs a package at runtime** — the failure is guaranteed.
- A package that is NOT in the list above can only be added by the **operator** baking it into the sandbox image (a one-line edit to `Dockerfile.sandbox` + rebuild on the VPS). If a task genuinely needs one, name the package and why, and let the user decide — then assume it is present only after they confirm the rebuild.
- **scrapy and other crawling frameworks are intentionally absent**: they assume crawling arbitrary hosts, and every hostname here needs an explicit operator grant. For fetch-and-parse against granted hosts, use `requests` + `bs4/lxml` (with `/state` for cursors and a schedule for recurring pulls, if the task needs them).

Network

Every program declares one of two modes in its manifest.

`none` (**default — prefer it**)

No network stack at all. Use this for everything that doesn't strictly need the internet. It has no failure modes and needs nothing from anyone.

`allowlist`

Outbound HTTP/HTTPS only, and only through the sandbox's proxy:

- The environment provides `HTTP_PROXY` / `HTTPS_PROXY` (and lowercase variants). **Only clients that honor proxy environment variables work.** Python's `urllib.request` honors them — use it. Node's built-in `fetch` ignores them and will not work for external hosts. For networked programs, python is the reliable choice.
- There is no DNS resolution inside the sandbox for external names; the proxy resolves the target itself. Connect by hostname through the proxy — never by IP, and never with a custom resolver.
- **Reachable hostnames are decided by the operator's global allowlist on the runner — not by the program's manifest.** The `allowed_hosts` a program declares is a *request*: it is shown to the user on the approval card so they can grant it on the runner. Until the user grants a host there, the proxy refuses it (typically a `403` on the tunnel), regardless of what the manifest says. The declared list is also mirrored to the program in the `CCPROG_ALLOWED_HOSTS` env var, for its own reference.
- A refused host surfaces as an ordinary connection error. The recommended pattern — which also classifies the run correctly — is to catch it and print the marker, then exit 0:

```
python import os, json, urllib.request try: body =
urllib.request.urlopen("https://example.com/", timeout=10).read()
print(body[:100].decode(errors="replace")) except Exception as e: print("ccprog: egress
blocked example.com", e) ``
```

- Internal and private addresses (the VPS itself, cloud metadata, other containers, private ranges) are unreachable in every mode.

Currently granted hosts (the user maintains this line as grants change): as of 2026-08-25 — `example.com`. To use a new service, name the exact hostname(s) to the user and ask them to grant it on the runner; it is a one-minute edit on their side.

Secrets (API tokens etc.)

- A program's manifest lists secret **names** (e.g. `HERMES_TOKEN`). Values live only in the runner's config on the VPS, entered by the user. At run time, each declared name that exists there is injected as an environment variable into that one container.
- Chat, the app, and forge-time never see the values, and neither do you — write programs that read them from the environment by name.
- Output redaction removes only the literal value string; anything a program prints should be assumed visible to whoever reads the output. Don't print secret values or transformations of them.
- **Currently configured secrets:** none (as of 2026-08-25). If a program needs a token, tell the user the name to add on the runner.

Verification (the smoke test every draft runs)

Before approval, every draft executes **hermetically: network mode none and no secrets, regardless of the manifest**. The verdict is computed mechanically from the exit code and expected output. Consequences for how you write:

- Every program — including networked ones — must have a meaningful offline path that exits 0 and produces its expected output. For networked programs, the "blocked/offline" branch (marker + exit 0, as above) *is* the verified path.
- Make programs deterministic where the expected output is checked; put variable data (timestamps, fetched content) outside the checked portion.
- Handle missing `PROGRAM_ARGS`, missing secrets, and absent network without crashing — verification runs in exactly that stripped state.

Lifecycle facts

- Drafts are versioned by insertion — every edit is a new version. Approval pins the exact source; only pinned, approved code runs. If code must change, that is a new version needing a new approval.
- Approved programs are permanent, rerunnable capabilities, filed in the library for retrieval.

Long unattended runs (schedules)

- A **live** run is always capped at ~60 seconds. There is no way to start a longer run directly, and promising one is a lie — don't.
- The **user** can put an approved program on a schedule (Settings → Program Foundry) and grant that schedule a runtime allowance of **up to 1 hour**. Scheduled long runs execute unattended in the background; their results land in the run history a few minutes after they finish, not in live chat.
- Writing for a long scheduled run is the same discipline as any other program — read `PROGRAM_ARGS` defensively (schedules pass `{}`), print a compact result, exit 0 on success — plus: make progress durable in `/state` if the work is resumable, because a runner restart aborts an in-flight run (it is recorded as `lost`, and the next scheduled slot retries).
- When a task genuinely needs minutes of wall-clock (large pulls, slow APIs, batch processing), forge the program normally and tell the user what schedule + runtime allowance to set. Suggest splitting work across shorter runs first — it is usually the better design.

When the task needs more than the sandbox has

Make one clear, specific request to the user rather than iterating drafts:

- **New internet destination** → name the exact hostname(s) to grant.
- **API credential** → name the secret to add on the runner.
- **Third-party library** → name the package to bake into the sandbox image.
- **More than 60s of wall-clock** → ask the user to schedule it with a runtime allowance (up to 1 hour), or split the task across runs.
- **More memory / output than the limits above** → say what the task actually needs; the operator can tune runner limits.

Good fits / poor fits

Good: text and data processing, parsing and transformation, statistics, generation (names, tables, structured data), format conversion, calling one or two granted APIs with `stdlib HTTP`, self-contained computations that finish in seconds and print their result, recurring scheduled jobs that keep compact working state in `/state`.

Poor: anything needing heavy third-party frameworks at runtime, crawling arbitrary/many hosts (each hostname needs an operator grant), listening processes or anything that must outlive its run, bulk file storage (persist state is a ~256 MB working set, not an archive), results larger than the output cap.